

Relatório Projeto #3 AED 2025-2026

Nome: Tomás Simões Correia

Nº Estudante: 2024198175

PL (inscrição):

NOTA: máximo 3 páginas

Introdução

Este projeto avalia experimentalmente três algoritmos de ordenação — Bubble Sort, Merge Sort e Heap Sort, aplicados a dados de estudantes, cada um com número de identificação, tipo de curso (0: Licenciatura, 1: Mestrado, 2: Não Conferente de Grau) e classificação final (10–20). A otimização das listagens 2 e 3 não foi implementada.

O Bubble Sort troca repetidamente elementos adjacentes fora de ordem, terminando antecipadamente se nenhuma troca ocorrer numa passagem. É estável e tem complexidade $O(n)$ no melhor caso e $O(n^2)$ nos restantes.

O Merge Sort divide o array recursivamente, ordena cada metade e funde-as. É estável: em empate, o elemento da esquerda tem prioridade. Tem complexidade $O(n \log n)$ em todos os casos, mas requer memória auxiliar $O(n)$.

O Heap Sort constrói uma max-heap e extrai repetidamente o máximo. É instável, pois a construção da heap altera a ordem relativa de elementos iguais. Tem complexidade $O(n \log n)$ e memória auxiliar $O(1)$.

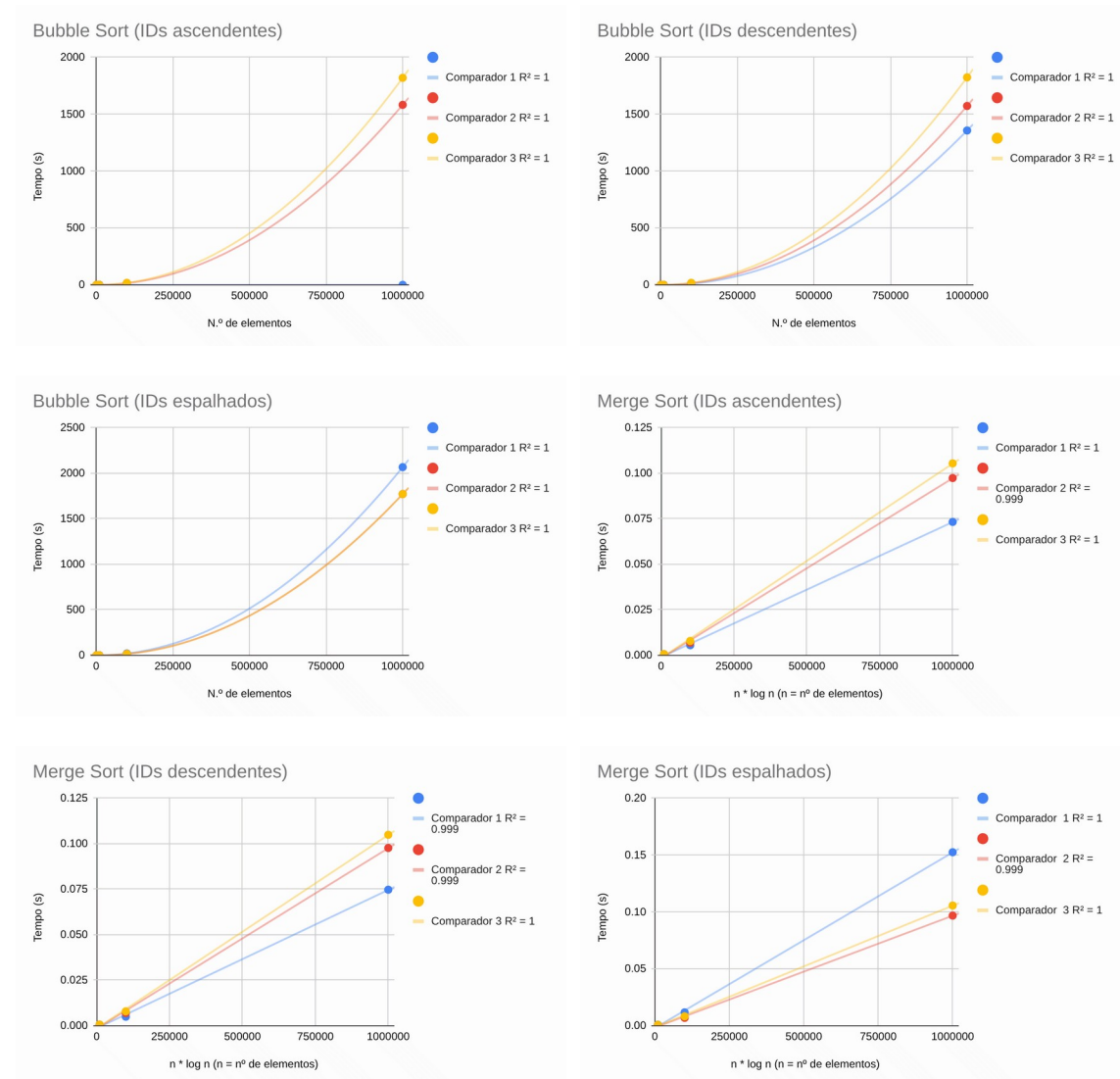
Implementação

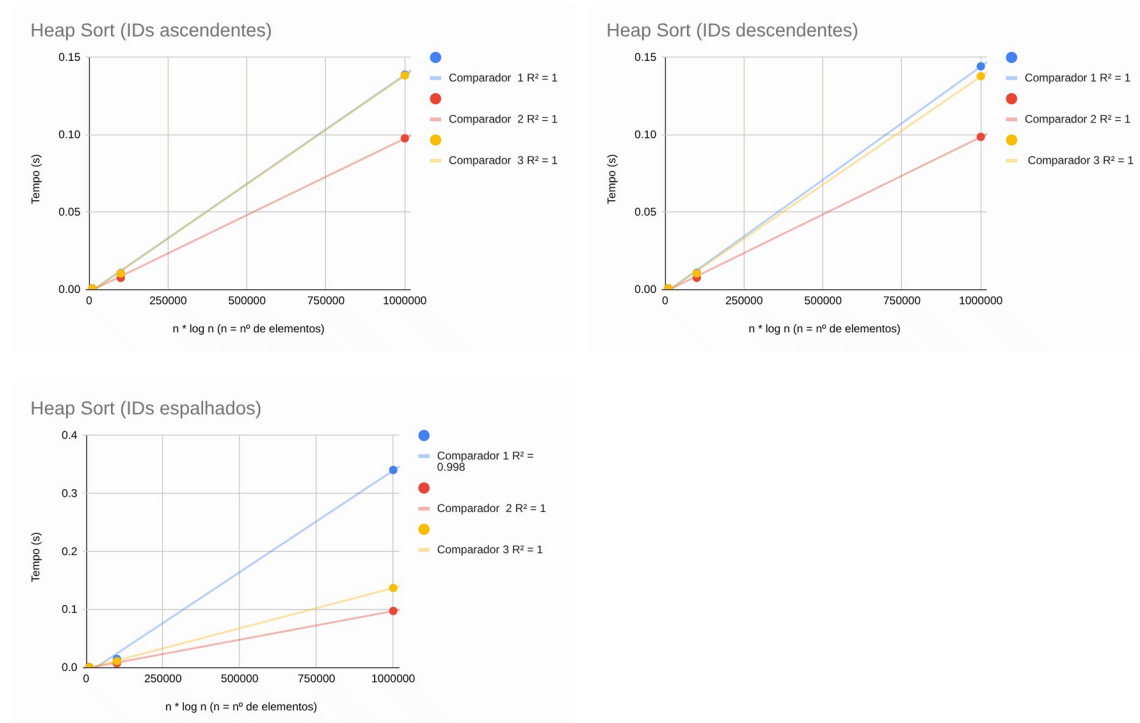
Todos os algoritmos recebem o array, índices de início e fim, e um comparador como ponteiro de função. No Bubble Sort, após cada passagem o índice de fim é decrementado, pois o maior elemento fica na posição correta e verifica-se se não foi efetuada nenhuma troca em cada iteração, para possivelmente terminar antecipadamente. No Merge Sort, os arrays auxiliares são alocados na heap com “malloc” para evitar stack overflows em arrays de grande dimensão. No Heap Sort, a função “heapify” é iterativa por eficiência, e os índices dos filhos são calculados relativamente ao índice de início, permitindo operar sobre subarrays arbitrários.

Geração de dados

São gerados conjuntos de IDs crescentes, decrescentes e aleatórios, para tamanhos 1 000, 10 000, 100 000 e 1 000 000, cada um repetido 3 vezes e guardado em CSV. Aproximadamente 5% dos IDs são repetidos 2 ou 3 vezes, com tipo de curso e classificação gerados independentemente. As classificações seguem uma Binomial(10, 1/3) deslocada de 10, com valor esperado $\approx 13,3$. Os IDs aleatórios são obtidos por permutação de Fisher-Yates sobre os IDs crescentes.

Avaliação experimental





Análise dos resultados

Os resultados confirmam as complexidades teóricas. O Bubble Sort é obviamente o mais lento, enquanto os outros dois algoritmos ordenam 1 000 000 elementos em frações de segundo, o Bubble Sort pode demorar até 2000 segundos, sendo impraticável para muitos dados.

O tipo de dados afeta significativamente o Bubble Sort com o Comparador 1 com IDs crescentes. O array já está quase ordenado e a saída antecipada reduz drasticamente o tempo. Com IDs decrescentes, todos os elementos estão na ordem inversa, atingindo o pior caso. Os Comparadores 2 e 3 ignoram o ID, pelo que o tipo de dados não tem impacto relevante no seu desempenho.

O Merge Sort e o Heap Sort apresentam tempos semelhantes, com crescimento proporcional a $O(n \log n)$ e pouca sensibilidade ao tipo de dados ou comparador. O Merge Sort é ligeiramente mais rápido geralmente, possivelmente devido a melhor localidade de memória.

Concluindo, o Merge Sort é a melhor escolha, combinando velocidade e estabilidade. O Heap Sort é uma alternativa válida quando a memória é limitada. O Bubble Sort só é viável para conjuntos pequenos ou já quase ordenados.